

SAPIENZA
UNIVERSITÀ DI ROMA

Title

Distributed System Notes

Professor:
Alessandro Mei

Student:
Carlo Da Roma

Academic Year 2026/2027

Contents

1 Introduction **3**

2 Asynchronous Distributed Systems **4**

3 Distributed Computations **4**

4 Global States, Cuts and Runs **6**

5 Monitoring Distributed Computations **7**

6 Consistency **8**

7 Observing Distributed Computations **9**

8 Vector Clocks **13**

9 Causal Message Order and Causal Delivery **14**

10 Constructing Global States and Snapshots **16**

11 Classification of Global Predicates **17**

12 Algorithms for Global Predicate Evaluation (GPE) **18**

13 7.1 Introduction **20**

13.1 Distributed Transaction Processing 20

13.2 The Distributed Commit Problem 20

13.3 Partial Failures and Atomic Commitment Protocols 21

14 7.2 Failures in a Distributed System **21**

14.1 Network and Site Architecture 21

14.2 Site Failure Model 21

14.3 Communication Failures and Network Partitions 21

14.4 Undeliverable Messages 22

14.5 Failure Detection via Timeouts 22

15 7.3 Atomic Commitment **22**

15.1 Conditions for Local Consent 22

15.2 Abstraction Model 22

15.3 Formal Correctness Conditions for ACPs 23

15.4 Uncertainty Period and Blocking 23

15.5 Fundamental Limits of ACPs 24

16 7.4 The Two Phase Commit Protocol (2PC) 24
16.1 Basic Protocol Execution (No Failures) 24
16.2 Timeout Actions 24
16.3 Termination Protocols 25
16.4 DT Log Management and Recovery Rules 25
16.4.1 Logging Steps during Execution 25
16.4.2 Site Restart Procedure 26
16.5 Garbage Collection of DT Log 26
16.6 Evaluation of Centralized 2PC 26

References 27

1 Introduction

Distributed Systems is a branch of computer science that focuses on designing, building, and managing systems composed of multiple autonomous computers (nodes) connected via a network, which coordinate their actions to operate as a single unified system to the end user.

The subject primarily addresses how to maintain reliability, scalability, and performance across distributed environments. Key topics covered include:

This field provides the theoretical and practical foundation for modern tech infrastructure, including cloud computing (AWS, GCP), distributed databases (Cassandra, Spanner), microservice architectures, and large-scale web applications.

The central problem addressed in the work by Babaoğlu and Marzullo is **Global Predicate Evaluation** (GPE) in asynchronous distributed systems. Many fundamental problems in distributed programming can be formulated as the need to detect when the global state of the system satisfies a specific Boolean condition Φ and react accordingly.

Practical applications of GPE include:

- **Monitoring and debugging** of distributed programs.
- **Detection of stable states**, such as deadlock, computation termination, token loss, and garbage collection.
- **Dynamic configuration management**, such as load balancing, checkpointing, and restarting.

Difficulties in Asynchronous Distributed Systems

In a distributed system, the global state Σ is defined as the union of the local states of individual processes. Since processes do not share memory and communicate solely through message passing, an observer process wishing to construct a global state must deduce remote components via messages.

The intrinsic uncertainty factors are:

1. **Unpredictable communication delays** and unbounded relative execution speeds.
2. **Temporal uncertainty**: a process cannot determine the instantaneous global state of the system.
3. **Inadequacy of high communication frequency**: increasing the number of messages makes local views more recent and complete, but in no way guarantees that the constructed global state is *consistent*.

4. **Relativistic Effect**: different distributed observers may construct different global states for the same computation due to variations in network delays. These states may all be consistent and evaluate the same predicate, but lead processes to execute incompatible or conflicting reactions.

—

2 Asynchronous Distributed Systems

A distributed system is formalized as a collection of n individual sequential processes $p_1, p_2, \ldots, p_n$ and a network of reliable unidirectional communication channels between pairs of processes.

Network Model Properties

- **Channel Reliability**: messages are not lost, duplicated, or corrupted.
- **Basic Non-FIFO**: channels may deliver messages in an order different from the send order.
- **Strong Connectivity**: every process can communicate with any other process, possibly through intermediate processes.

Asynchronous Model Assumptions

To establish upper bounds on the complexity of distributed problems (if a problem is solvable in the asynchronous model with cost C, it will be solvable with cost $\leq C$ in any real system), the system with the weakest possible assumptions is adopted:

- **No upper bound** on the relative execution speed of processes.
- **No upper bound** on message transmission delays in the network.
- **Absence of synchronized real-time clocks** or a shared global real time.
- **Exclusive synchronization via messages**: message passing is the only available mechanism to synchronize activities.

—

3 Distributed Computations

A distributed computation models the execution of a distributed program by a collection of processes. The activity of each sequential process p_i is modeled as an ordered sequence of events.

Event Types

1. **Internal Events**: only cause a change in the local state of process p_i.
2. **Communication Events**:
   - Send event $send_i(m)$: inserts message m into the outgoing channel toward the destination process.
   - Receive event $receive_j(m)$: extracts message m from the incoming channel of process p_j.

Each message m possesses a unique identifier (for instance, the pair formed by process ID and sequence number) that uniquely pairs the $send_i(m)$ event to the $receive_j(m)$ event. The event reception requires both the arrival of the message at the channel and the process's readiness to receive it.

Local Histories and Global History

- **Local History (h_i)**: total sequence of events executed by p_i:

$$h_i = e_i^1, e_i^2, e_i^3, \ldots$$

  The index k in e_i^k represents the **canonical enumeration**.

- **Local History Prefix (h_i^k)**: sequence of the first k events of p_i, with $h_i^0 = \emptyset$.
- **Global History (H)**: the set of all events in the computation:

$$H = h_1 \cup h_2 \cup \cdots \cup h_n$$

Lamport's Causal Precedence Relation ($\rightarrow$)

The relative order of events in an asynchronous system is established exclusively by the cause-and-effect relationship (information flow) $\rightarrow$, defined as the smallest partial order on H satisfying three conditions:

1. If $e_i^k, e_i^l \in h_i$ with $k < l$, then $e_i^k \rightarrow e_i^l$.
2. If $e = send(m)$ and $e' = receive(m)$, then $e \rightarrow e'$.
3. If $e \rightarrow e''$ and $e'' \rightarrow e'$, then $e \rightarrow e'$ (transitive closure).

If two events e and e' are not causally related, i.e., $\neg(e \rightarrow e') \wedge \neg(e' \rightarrow e)$, they are said to be **concurrent**, denoted by $e \parallel e'$.

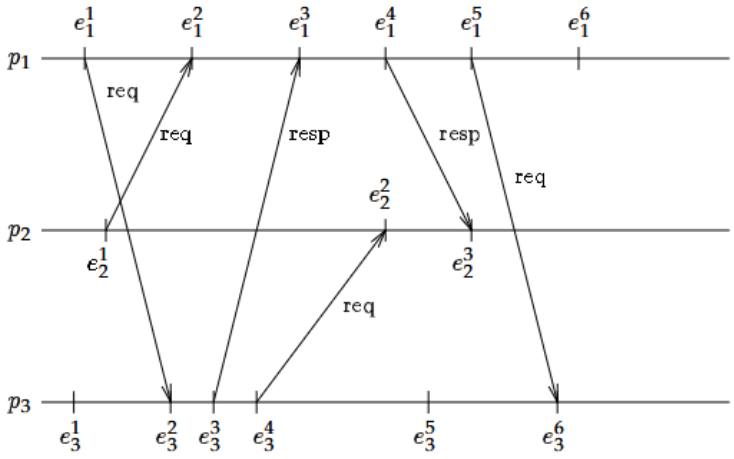


Figure 1: *Diagram 1: Example of a Space-Time Diagram. A directed path from left to right along process lines and message arrows determines causal precedence* $e \to e'$.

Poset and Space-Time Diagram

A distributed computation is formally defined as a **partially ordered set** (poset) $(H, \to)$.

—

4 Global States, Cuts and Runs

Definitions of Global State and Cut

- **Local State** (σ_i^k): the state of process p_i immediately following the execution of event e_i^k (σ_i^0 denotes the initial state).
- **Global State** (Σ): an n-tuple of local states, one for each process:

$$\Sigma = (\sigma_1, \sigma_2, \ldots, \sigma_n)$$

- **Cut** (C): a subset of the global history H containing an initial prefix of the local history of each process:

$$C = h_1^{c_1} \cup h_2^{c_2} \cup \cdots \cup h_n^{c_n}$$

  A cut is uniquely identified by the tuple of indices $(c_1, c_2, \ldots, c_n)$.

- **Cut Frontier**: the set of the last events included in the cut for each process: $(e_1^{c_1}, e_2^{c_2}, \ldots, e_n^{c_n})$. The global state corresponding to the cut is $(\sigma_1^{c_1}, \sigma_2^{c_2}, \ldots, \sigma_n^{c_n})$.

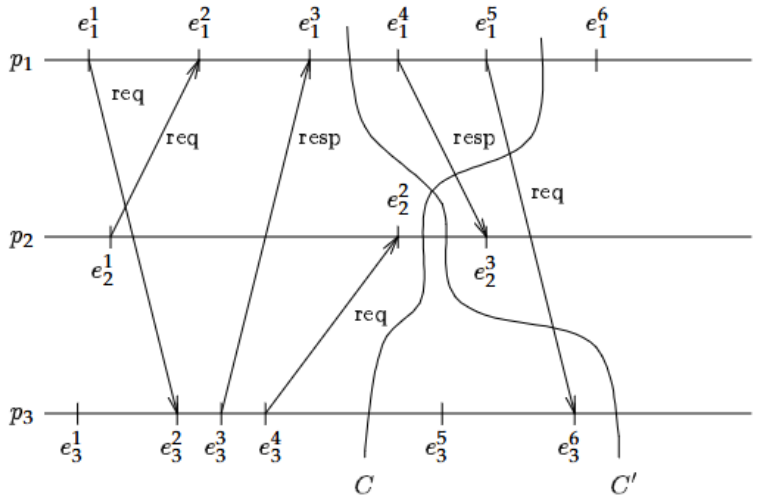


Figure 2: Cuts of a Distributed Computation

Execution (Run R)

An **execution** (Run) R of a distributed computation is a total ordering of all events in H that respects the local ordering of each history h_i. A single distributed computation $(H, \rightarrow)$ admits multiple different runs, each corresponding to a different sequential execution sequence.

—

5 Monitoring Distributed Computations

In studying Global Predicate Evaluation (GPE), the presence of a single monitor process p_0 is initially assumed.

Active Monitoring Strategy

In active monitoring, p_0 periodically sends a state enquiry message to all processes $p_1, \ldots, p_n$. Upon receiving the request, each process p_i responds by sending its local state σ_i to p_0. p_0 assembles these states to form a global state $(\sigma_1, \ldots, \sigma_n)$.

The Ghost Deadlock Problem

Consider a client-server system where servers maintain a Waits-For Graph (WFG^+), where a directed edge exists from p_i to p_j if p_j has received a request from p_i to which it has not yet responded. The deadlock predicate is $\Phi =$ "WFG^+ contains a cycle".

If p_0 collects local states at moments corresponding to an inconsistent cut (such as cut $C'' = (3, 2, 6)$), it may collect a state from p_3 recording the reception of a request

sent by p_1, while the state collected from p_1 precedes the sending event of that request. The resulting WFG^+ will display a fictitious cycle. This condition is termed **Ghost Deadlock**: the monitor detects a deadlock that never existed in the real system.

Diagram 2: Representation of an Inconsistent Cut C''. The message is received to the right of the frontier but sent to the left.

—

6 Consistency

Formal Definition of Consistent Cut

Causal precedence provides the mathematical formalism to define consistency. A cut C is defined as **consistent** if and only if it is left-closed with respect to the causal precedence relation $\rightarrow$:

$$\forall e, e' \in H : (e' \in C \land e \rightarrow e') \implies e \in C$$

Graphical Rule: A cut is consistent if and only if in the space-time diagram no message arrow crosses the cut frontier from right to left (i.e., with its head to the left and tail to the right of the frontier).

A **consistent global state** is the global state associated with a consistent cut. They represent all and only the global states that could have been observed by an ideal external observer.

Consistent Runs and the Lattice of Global States

An execution R is defined as **consistent** if it is a linear extension of the partial order defined by $\rightarrow$, i.e.:

$$\forall e, e' \in H : (e \rightarrow e') \implies e \text{ appears before } e' \text{ in } R$$

A consistent run generates a sequence of consistent global states $\Sigma^0, \Sigma^1, \Sigma^2, \ldots$. Defining the *leads-to* relation ($\sim_R$), where $\Sigma^{k-1} \sim_R \Sigma^k$ indicates that state Σ^k is obtained from Σ^{k-1} by executing a single event e^k, the set of all consistent global states and the relation $\sim$ form a **lattice**.

- **Lattice Axes**: n orthogonal axes, one for each process.
- **Global State Level**: $\text{Level}(\Sigma) = \sum_{i=1}^{n} c_i$ for a state $\Sigma = (\sigma_1^{c_1}, \ldots, \sigma_n^{c_n})$.
- **Lattice Path**: Every directed path from the origin $\Sigma^{0\ldots0}$ toward increasing levels corresponds to a possible consistent run of the computation.

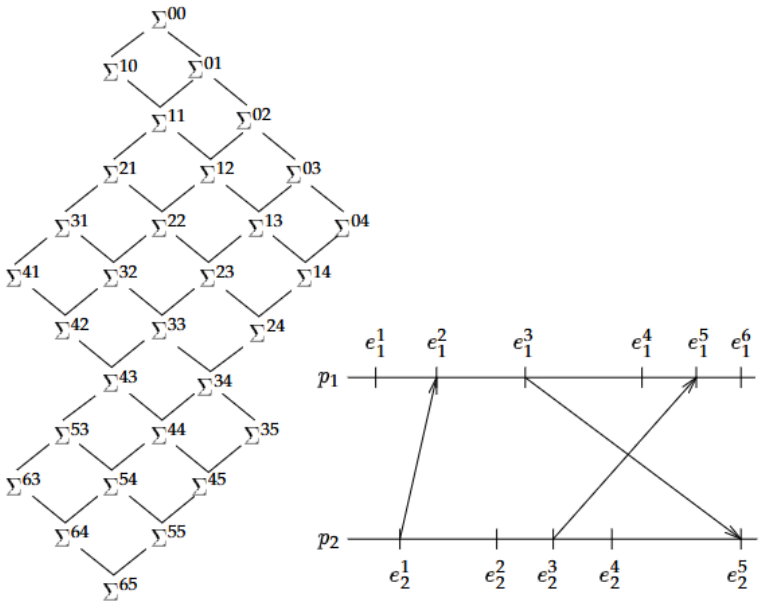


Figure 3: A Distributed Computation and the Lattice of its Global States

—

7 Observing Distributed Computations

Passive / Reactive Architecture

In a passive architecture, the monitor p_0 does not send enquiry messages. Application processes notify p_0 of the execution of each relevant event via notification messages. The arrival sequence of these notifications at p_0 constitutes an **observation** of the computation.

An observation may correspond to a consistent run, an inconsistent run, or not correspond to any run (for instance, if events from the same process are received out of canonical order).

FIFO Delivery Rule

To prevent notifications from the same process from arriving out of order, a `deliver` primitive distinct from the network `receive` primitive is introduced. **FIFO** delivery guarantees that:

$$\forall m_1, m_2 : send_i(m_1) \rightarrow send_i(m_2) \implies deliver_j(m_1) \text{ precedes } deliver_j(m_2)$$

FIFO delivery eliminates observations that do not correspond to runs, but **does not guarantee the consistency of the observation** if it involves different senders.

Synchronous Model with Real Time (Rule DR1)

Temporarily assuming the presence of synchronized global real-time clocks RC and a maximum message delay bounded by δ, the clock condition is defined:

$$e \to e' \implies RC(e) < RC(e')$$

Delivery Rule DR1:

> At real time t, monitor p_0 delivers all received messages carrying timestamp $RC \leq t - \delta$, ordered by increasing timestamp.

Rule DR1 guarantees that the observation is consistent because waiting for time δ ensures that no future messages with timestamps lower than those delivered can arrive.

—

Logical Clocks

In asynchronous systems lacking real time, Lamport introduced **logical clocks**. Each process p_i maintains a local logical clock LC_i assigning positive integers to events.

Logical Clock Update Rules

All processes initialize $\mathrm{LC}_i := 0$. For each new event e_i occurring at process p_i:

1. If e_i is an internal event or a send event send(m):

   $$\mathrm{LC}_i(e_i) := \mathrm{LC}_i + = 1$$

   (this means that also

   $$\mathrm{LC}_i$$

   encrease its value)

2. If e_i is a receive event $e_i = \text{receive}(m)$, with message timestamp $\mathrm{TS}(m) = \mathrm{LC}_j(\text{send}(m))$:

   $$\mathrm{LC}_i(e_i) := \max(\mathrm{LC}_i, \mathrm{TS}(m)) + 1$$

Logical clocks satisfy the **Clock Condition**:

$$e \to e' \implies \mathrm{LC}(e) < \mathrm{LC}(e')$$

The Gap-Detection Problem and Stability

Merely checking $LC(e) < LC(e')$ **does not imply** $e \rightarrow e'$ (they could be concurrent). Furthermore, logical clocks lack the **Gap-Detection** property:

> **Gap-Detection**: Given two events e and e' with $LC(e) < LC(e')$, determine whether or not there exists an event e'' such that $LC(e) < LC(e'') < LC(e')$.

Without bounds on message delays, p_0 cannot know if a notification with a lower logical timestamp will arrive in the future.

Stable Messages and Delivery Rule DR2

Assuming FIFO channels between each application process and monitor p_0, a message m received by p_0 is defined as **stable** if p_0 has received at least one message with a timestamp greater than $TS(m)$ from *every* other process in the system.

Delivery Rule DR2:

> Monitor p_0 delivers notified messages that are **stable**, strictly ordered by increasing logical clock timestamp.

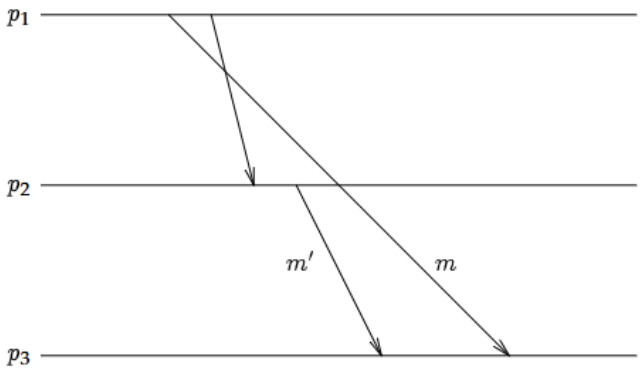


Figure 4: Message Delivery that is FIFO but not Causal

Because channels are FIFO and local logical clocks are strictly increasing, stability ensures that no message with a logical timestamp lower than or equal to that of the delivered message will ever arrive, guaranteeing a consistent observation of the system.

—

Comparative Table of Delivery Rules (s 7 and 8)

—

Rule	Assumed Model	Mechanism	Delivery / Stability Condition
DR1	Synchronous (Delay $\leq \delta$)	Global Real Clocks (RC)	Delivers at time t all messages with $RC \leq t - \delta$.
DR2	Asynchronous with FIFO channels to p_0	Lamport's Logical Clocks (LC)	Delivers stable messages (received a $TS > TS(m)$ from every process).

Comparison Table of Monitoring Strategies (s 5 and 7)

Feature	Active Monitoring (Snapshots)	Passive Monitoring (Reactive)
Monitor Role (p_0)	Active: sends state enquiry messages.	Passive: receives event notifications only.
Communication Overhead	Generated only when executing snapshots.	Continuous for each relevant event.
Construction Result	Single global state Σ (a cut).	Total sequence of states (an observation / run).
Inconsistency Risk	Ghost Deadlock risk if cut is inconsistent.	Guaranteed consistent if paired with DR1 or DR2.

8 Vector Clocks

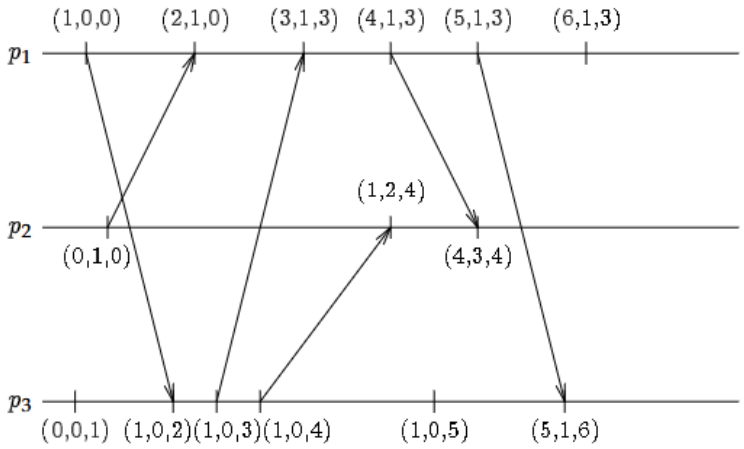


Figure 5: Vector Clocks

Limitations of Scalar Logical Clocks

Lamport's scalar logical clocks satisfy the *Clock Condition*:

$$e \to e' \implies LC(e) < LC(e')$$

However, the converse does not hold:

$$LC(e) < LC(e') \not\Rightarrow e \to e'$$

Because scalar clocks collapse multi-dimensional execution histories into a single integer, they cannot distinguish whether two events are causally dependent ($e \to e'$) or concurrent ($e \parallel e'$). To characterize the exact causal structure of a distributed computation, a clock mechanism with strong causal isomorphism is required.

Definition and Update Rules for Vector Clocks

Vector clocks (developed independently by Fidge and Mattern) assign an n-dimensional vector of non-negative integers V_i to each process p_i in a system of n processes. The entry $V_i[j]$ represents process p_i's knowledge of the number of events executed by process p_j.

Vector Clock Protocol

1. **Initialization**: All processes p_i initialize their vector clocks to zeros:

$$V_i[j] := 0 \quad \forall j \in \{1, \ldots, n\}$$

2. **Local/Send Event**: Before executing an internal event or a send event, process p_i increments its own component:

$$V_i[i] := V_i[i] + 1$$

3. **Message Transmission**: When p_i sends a message m, it attaches its current vector timestamp $V_m = V_i$ to m.

4. **Message Reception**: When process p_j receives a message m carrying timestamp V_m, it updates its vector component-wise by taking the maximum and then increments its local entry:

$$\forall k \in \{1, \ldots, n\} : V_j[k] := \max(V_j[k], V_m[k])$$

$$V_j[j] := V_j[j] + 1$$

Vector Comparison Rules and Properties

Given two vector timestamps V and V' of size n:

- **Equality**: $V = V' \iff \forall k \in \{1, \ldots, n\} : V[k] = V'[k]$
- **Less Than or Equal**: $V \leq V' \iff \forall k \in \{1, \ldots, n\} : V[k] \leq V'[k]$
- **Strict Precedence**: $V < V' \iff (V \leq V') \wedge (V \neq V')$
- **Concurrency**: $V \parallel V' \iff \neg(V \leq V') \wedge \neg(V' \leq V)$

Isomorphism Theorem

Vector clocks establish an exact equivalence (isomorphism) with Lamport's causal precedence relation:

$$e \to e' \iff V(e) < V(e')$$

$$e \parallel e' \iff V(e) \parallel V(e')$$

Diagram 4: Progression of Vector Clocks across three communicating processes.

—

9 Causal Message Order and Causal Delivery

Definition of Causal Order (CO)

Causal order is a fundamental communication abstraction ensuring that message delivery respects the causal precedence of message transmissions. Formally, if the sending of message m_1 causally precedes the sending of message m_2:

$$send_i(m_1) \to send_k(m_2)$$

then every process p_j that delivers both messages must deliver m_1 before delivering m_2:

$$deliver_j(m_1) \text{ precedes } deliver_j(m_2)$$

Distinction Between FIFO and Causal Order

- **FIFO Order**: Ensures that messages sent by the *same* process arrive in the order they were sent. It provides no constraints across different senders.
- **Causal Order**: Enforces ordering across *multiple* senders whenever message transmissions are causally related via intermediate interactions.

Causal Delivery Protocol Using Vector Clocks (CBCAST)

To enforce Causal Order without central coordination, processes attach vector timestamps to messages.

Each process p_i maintains a vector clock V_i, where $V_i[k]$ counts the number of messages sent by p_k that have been delivered to p_i.

Delivery Conditions

When process p_j receives a message m sent by process p_i with attached timestamp V_m, process p_j delays the delivery of m until two conditions are met:

1. $V_m[i] = V_j[i] + 1$: m is the immediate next message expected from sender p_i.
2. $\forall k \neq i : V_m[k] \leq V_j[k]$: Process p_j has already delivered all messages from other processes that causally preceded the sending of message m.

When these conditions are satisfied, p_j delivers m and updates its local vector clock entry:

$$V_j[i] := V_j[i] + 1$$

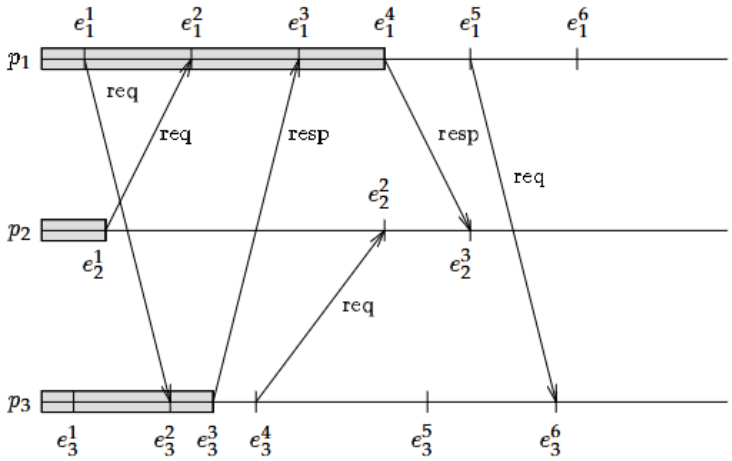


Figure 6: Causal History of Event e(4,1)

—

10 Constructing Global States and Snapshots

The Global Snapshot Problem

In an asynchronous system, taking a snapshot requires capturing the local states of all processes along with the states of all communication channels such that the combined global state is **consistent** (i.e., corresponds to a valid cut).

The Chandy-Lamport Snapshot Algorithm

The Chandy-Lamport algorithm records a consistent global state without halting execution. It assumes channels are unidirectional, reliable, and strictly FIFO.

It uses a special control message called a **Marker**.

Algorithm Rules

- **Marker Sending Rule for Process** p_i:
  1. Process p_i records its current local state σ_i.
  2. For each outgoing channel c, p_i sends one marker along c before sending any further application messages on c.
- **Marker Receiving Rule for Process** p_j **on Incoming Channel** c:
  - **If** p_j **has not yet recorded its local state**:

1. p_j records its local state σ_j.
2. p_j records the state of channel c as empty ($\emptyset$).
3. p_j executes the Marker Sending Rule.

– **If p_j has already recorded its local state**:

1. p_j records the state of channel c as the sequence of all messages received on c after p_j recorded its state and before p_j received the marker on c.

Properties and Correctness

Let Σ_i be the global state in which the snapshot algorithm was initiated, and Σ_f be the global state when the snapshot terminates.

The recorded snapshot state Σ_{snap} is guaranteed to be a consistent global state satisfying:

$$\Sigma_i \rightsquigarrow \Sigma_{snap} \rightsquigarrow \Sigma_f$$

where $\rightsquigarrow$ denotes reachability through a valid execution run. While Σ_{snap} may never have existed at a single real-time instant, it represents a valid linear extension of the partial order.

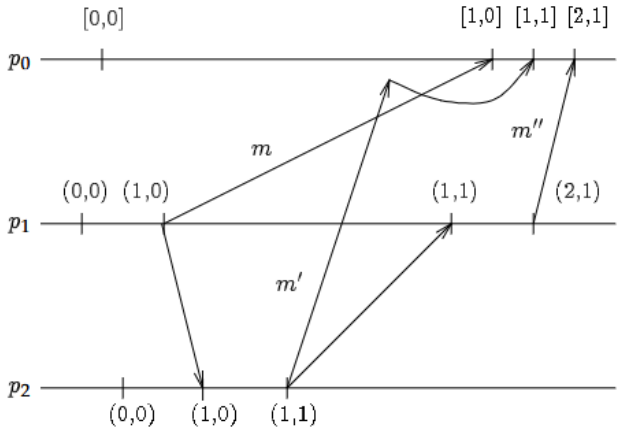


Figure 7: Causal Delivery Using Vector Clocks

—

11 Classification of Global Predicates

Evaluating a global predicate Φ involves determining whether a Boolean condition on the global state holds during execution.

Stable vs. Non-Stable Predicates

- **Stable Predicates**: A predicate Φ is stable if once it becomes true in a consistent global state Σ, it remains true in all consistent global states Σ' reachable from Σ:

$$\Phi(\Sigma) \wedge (\Sigma \rightsquigarrow \Sigma') \implies \Phi(\Sigma')$$

  Examples: Deadlock, computation termination, token loss, garbage collection.

- **Non-Stable Predicates**: A predicate Φ that can fluctuate between true and false arbitrarily throughout execution. *Examples*: Mutual exclusion violations, variable value bounds, buffer overflow conditions.

Modalities for Non-Stable Predicates over the Lattice

Because an asynchronous computation corresponds to a lattice $\mathcal{L}$ of consistent global states with multiple valid execution paths (runs), non-stable predicates are interpreted using temporal modalities.

1. Possibly(Φ) (or Weakly(Φ))

Possibly(Φ) is true if there exists at least one consistent run R passing through a global state Σ where $\Phi(\Sigma)$ holds:

$$\text{Possibly}(\Phi) \iff \exists R \in \mathcal{L}, \exists \Sigma \in R : \Phi(\Sigma) = \text{true}$$

2. Definitely(Φ) (or Strongly(Φ))

Definitely(Φ) is true if *every* consistent run R passes through at least one global state Σ where $\Phi(\Sigma)$ holds:

$$\text{Definitely}(\Phi) \iff \forall R \in \mathcal{L}, \exists \Sigma \in R : \Phi(\Sigma) = \text{true}$$

Diagram 5: Lattice paths illustrating state modalities. If only Σ_{01} satisfies Φ, Possibly(Φ) is true, but Definitely(Φ) is false because the path through Σ_{10} bypasses Φ.

—

12 Algorithms for Global Predicate Evaluation (GPE)

Evaluating Stable Predicates

Stable predicates can be evaluated using repeated global snapshot algorithms (such as Chandy-Lamport). If $\Phi(\Sigma_{snap}) = \text{true}$, the monitor safely concludes that Φ currently holds and will continue to hold indefinitely.

Evaluating Non-Stable Predicates: Lattice Construction

To evaluate Possibly(Φ) and Definitely(Φ), a central monitor process p_0 collects local state updates with vector timestamps from all processes and dynamically constructs the lattice of consistent global states level by level.

Lattice Traversal Algorithms (Marzullo-Neiger / Cooper-Marzullo)

- **Algorithm for Possibly**(Φ): Performs a Breadth-First Search (BFS) or Depth-First Search (DFS) over the constructed lattice. It terminates and returns `TRUE` immediately upon encountering the first global state Σ where $\Phi(\Sigma) = \text{true}$. If the entire lattice is explored without finding such a state, it returns `FALSE`.
- **Algorithm for Definitely**(Φ): Prunes paths in the lattice as soon as a state satisfying Φ is found. If all paths in the lattice are eventually pruned, it returns `TRUE`. If any path reaches the final global state without encountering a state where Φ holds, it returns `FALSE`.

Complexity Limitation

In general, constructing the lattice requires exponential time and space complexity:

$$O(k^n)$$

where n is the number of processes and k is the maximum number of events per process.

Efficient Special Cases: Weak Conjunctive Predicates (WCP)

When Φ can be decomposed into a conjunction of local predicates:

$$\Phi = \phi_1 \wedge \phi_2 \wedge \cdots \wedge \phi_n$$

where ϕ_i depends exclusively on the local state of process p_i, the evaluation complexity can be dramatically reduced.

Garg-Waldecker Algorithm for WCP

Instead of constructing the complete lattice, the Garg-Waldecker algorithm greedily advances candidate cuts. If a local predicate ϕ_i is false at state $\sigma_i^{c_i}$, no consistent global state containing $\sigma_i^{c_i}$ can satisfy Φ. Process p_i's candidate index c_i is immediately advanced, bypassing exponential state exploration.

- **Time Complexity**: $O(n^2 \cdot k)$, which is polynomial in the number of processes and events.
- **Space Complexity**: $O(n \cdot k)$ to store candidate vectors.

—

Summary Comparison of Predicate Modalities and Evaluation Methods

Category	Predicate Type	Quantification over Runs	Algorithmic Complexity
Stable	Deadlock, Termination	Hold in all reachable states after occurrence	O(Snapshot Cost)
Possibly(Φ)	Non-Stable / General	$\exists R \in \mathcal{L}, \exists \Sigma \in R : \Phi(\Sigma)$	$O(k^n)$ general lattice, $O(n^2k)$ for WCP
Definitely(Φ)	Non-Stable / General	$\forall R \in \mathcal{L}, \exists \Sigma \in R : \Phi(\Sigma)$	$O(k^n)$ state space traversal

13 7.1 Introduction

13.1 Distributed Transaction Processing

In a distributed database management system (DBS) without data replication, each data item is stored at a single unique site, managed by a local scheduler and Data Manager (DM) . A distributed transaction T originates at its home site, where operations are submitted to the local Transaction Manager (TM) .

Operation execution follows two distinct patterns:

- **Read and Write Operations**: Forwarded by the home TM to the site storing the target data item . Processed locally by that site's scheduler and DM as a local operation, returning the result to the home TM .
- **Commit and Abort Operations**: Unlike data access operations, a `Commit` or `Abort` affects every site where T accessed data . The home TM must distribute the termination request to all involved sites .

13.2 The Distributed Commit Problem

A transaction is committed not merely when the home TM issues a `Commit` message, but when the DM at every involved site successfully executes the `Commit` operation . If a local scheduler rejects a `Commit` request and aborts T, the transaction must be aborted across all participating sites to maintain global consistency .

13.3 Partial Failures and Atomic Commitment Protocols

Centralized systems exhibit binary failure behavior (either fully operational or entirely down) . Distributed systems experience **partial failures**, where a subset of sites or communication links fail while others remain operational .

The primary challenge in non-replicated distributed transaction processing is achieving **consistent termination**: ensuring that a single logical decision (`Commit` or `Abort`) is executed uniformly across all participating sites despite partial failures . An algorithm enforcing this requirement is called an **Atomic Commitment Protocol (ACP)** .

—

14 7.2 Failures in a Distributed System

14.1 Network and Site Architecture

A distributed system is modeled as a connected graph where nodes represent processing sites and undirected edges represent bidirectional communication links . Routing services are assumed to be provided by the underlying computer network layer .

14.2 Site Failure Model

Sites operate under the **fail-stop** assumption:

- A site is either functioning correctly (operational) or completely stopped (down) .
- Sites never perform incorrect, malicious, or arbitrary operations (no Byzantine behavior) .
- Volatile storage is lost during a failure; upon recovery, a site executes a `Restart` procedure utilizing stable storage to restore consistency .
- System states are categorized as **partial failures** (some sites operational, some down) or **total failures** (all sites down) . Operational sites in a partial failure may become uncertain regarding the state of failed sites, leading to potential blocking .

14.3 Communication Failures and Network Partitions

Communication channels are subject to transient or persistent disruptions:

- Lower network layers handle message corruption via error detection/retransmission and loss via link rerouting .

- A **network partition** occurs when a combination of site and link failures eliminates all operational communication paths between groups of sites, dividing the network into two or more disjoint operational components .

14.4 Undeliverable Messages

Messages become undeliverable if the destination site is down or partitioned . The system adopts a **dropped message policy** (undeliverable messages vanish) rather than persistent message storage, as persistent delivery protocols themselves require complex commitment mechanisms . Explicit nondelivery notifications are omitted due to inherent unreliability and infinite recursion risks .

14.5 Failure Detection via Timeouts

Sites detect communication unreachability through timeout mechanisms . A site A sends a message to site B and waits for a reply within a preset timeout period τ, which encompasses round-trip transmission and processing delays .

If τ elapses without a response, A concludes it cannot communicate with B . Underestimating τ or clock drift causes **timeout failures** (or performance failures), leading sites to falsely infer communication breakdown .

—

15 7.3 Atomic Commitment

15.1 Conditions for Local Consent

Before issuing a `Commit`, the TM must collect explicit consent from the local engines at every site S_i where T executed :

- **Scheduler Consent**: Granted if T satisfies the local recoverability condition (i.e., reads data written only by committed transactions) . Strictly serializable or cascadeless schedulers satisfy this continuously .
- **Data Manager Consent**: Granted if T satisfies the **Redo Rule** (all updates written by T are stored in stable storage—log or database) . Read-only sites do not require DM consent .

15.2 Abstraction Model

To analyze ACPs independently of specific engine mechanics, the execution model abstracts transaction processes into:

- **Coordinator**: The process at T's home site supervising the protocol (knows all participants) .
- **Participants**: Processes at all other sites where T executed (know the coordinator) .
- **Distributed Transaction Log (DT Log)**: Stable storage maintained at each site to record ACP state transitions across system restarts .

15.3 Formal Correctness Conditions for ACPs

In an ACP, each process casts a vote (`Yes` or `No`) and reaches a final decision (`Commit` or `Abort`) . The protocol must satisfy five correctness rules:

- **AC1 (Consistency)**: All processes that reach a decision must reach the exact same decision .
- **AC2 (Irrevocability)**: A process cannot change its decision once reached .
- **AC3 (Unanimity)**: The `Commit` decision can only be reached if every process voted `Yes` .
- **AC4 (Liveness)**: In an execution with no failures where all processes vote `Yes`, the decision must be `Commit` .
- **AC5 (Termination)**: In any execution with tolerable failures, if all existing failures are repaired and no new failures occur for long enough, all operational processes will eventually reach a decision .

15.4 Uncertainty Period and Blocking

- **Unilateral Abort**: Prior to voting `Yes`, any process can unilaterally decide `Abort` .
- **Uncertainty Period**: The interval between the moment a process votes `Yes` and the moment it receives sufficient information to learn the final global decision . While uncertain, a process cannot decide unilaterally .
- **Blocking**: Occurs when a process in its uncertainty period cannot communicate with other processes due to failures and must wait indefinitely for failure repair before reaching a decision .
- **Independent Recovery**: The ability of a recovering process to reach a decision without communicating with other sites .

15.5 Fundamental Limits of ACPs

Proposition 7.1: If communication failures or total site failures are possible, every ACP may cause processes to become blocked .

Proposition 7.2: No ACP can guarantee independent recovery of failed processes .

—

16 7.4 The Two Phase Commit Protocol (2PC)

16.1 Basic Protocol Execution (No Failures)

The standard Centralized 2PC protocol operates in two phases :

1. **Phase 1: Voting Phase**
   - The coordinator sends a `VOTE-REQ` message to all participants .
   - Upon receiving `VOTE-REQ`, a participant replies with its vote: `YES` or `NO` . If voting `NO`, the participant unilaterally decides `Abort` and stops .
2. **Phase 2: Decision Phase**
   - The coordinator collects all votes . If all participants (and the coordinator) voted `YES`, the coordinator decides `Commit` and broadcasts `COMMIT` messages .
   - If any vote was `NO`, the coordinator decides `Abort` and broadcasts `ABORT` messages to all participants that voted `YES` .
   - Participants receiving `COMMIT` or `ABORT` record the decision and stop .

The coordinator has no uncertainty period (decides immediately upon voting with full vote knowledge) . Participants enter their uncertainty period upon sending `YES` .

16.2 Timeout Actions

Timeouts prevent indefinite waiting during message steps :

- **Participant waiting for** `VOTE-REQ`: Has not voted yet → Unilaterally decides `Abort` and stops .
- **Coordinator waiting for votes**: Has not made a decision → Decides `Abort` and sends `ABORT` to all participants that sent `YES` .
- **Participant waiting for Decision (**`COMMIT/ABORT`**)**: Participant is in its uncertainty period → Must invoke a **Termination Protocol** .

16.3 Termination Protocols

- **Simple Termination Protocol**: The uncertain participant remains blocked until direct communication with the coordinator is restored .
- **Cooperative Termination Protocol**:
  1. The coordinator attaches a complete list of all participants to the initial `VOTE-REQ` message .
  2. An uncertain participant p timing out sends a `DECISION-REQ` message to all other processes q .
  3. A responder q acts as follows:
     - If q has decided `Commit` (or `Abort`), it sends `COMMIT` (or `ABORT`) to p .
     - If q has not voted yet, q unilaterally decides `Abort`, logs it, and sends `ABORT` to p .
     - If q is also in its uncertainty period, it cannot assist p (skips) .
  4. If all reachable processes are uncertain, p remains blocked until communication with a decided/unvoted process (or coordinator) is re-established .

16.4 DT Log Management and Recovery Rules

To meet condition AC5, state changes must be force-written to the DT log on stable storage :

16.4.1 Logging Steps during Execution

1. **Coordinator**: Writes a `start-2PC` record containing participant identities before or after sending `VOTE-REQ` .
2. **Participant Voting** `YES`: Force-writes a `yes` record (containing coordinator and participant lists) **before** transmitting `YES` .
3. **Participant Voting** `NO`: Writes an `abort` record before or after sending `NO` .
4. **Coordinator Decision**: Force-writes a `commit` record **before** sending `COMMIT` messages, or writes an `abort` record before/after sending `ABORT` messages .
5. **Participant Final Decision**: Writes a `commit` or `abort` record upon receiving the decision message .

16.4.2 Site Restart Procedure

When a site recovers from a failure, it inspects its local DT log :

- **If `start-2PC` record exists (Site was Coordinator)**:
  - Log contains `commit` or `abort` → Decision was made prior to crash; resume decision state .
  - Log contains neither → Unilaterally decide `Abort`, force-write an `abort` record, and send `ABORT` to participants .
- **If NO `start-2PC` record exists (Site was Participant)**:
  - Log contains `commit` or `abort` → Decision already reached prior to crash .
  - Log contains NO `yes` record → Failed before voting or voted `NO`; unilaterally decide `Abort` and force-write `abort` record .
  - Log contains a `yes` record but NO decision record → Failed during uncertainty period; invoke the termination protocol .

16.5 Garbage Collection of DT Log

To prevent infinite log growth, DT log records must be safely purged according to two principles :

- **GC1**: A site cannot delete log records of transaction T until its local Resource/Data Manager has completed `RM-Commit(T)` or `RM-Abort(T)` .
- **GC2**: At least one site (typically the coordinator) must retain records of T until it receives acknowledgments confirming that `RM-Commit(T)` or `RM-Abort(T)` has finished across all participating sites .

16.6 Evaluation of Centralized 2PC

- **Resiliency**: Tolerates site failures and communication failures (partitions, timeout/performance failures) .
- **Blocking**: Prone to blocking if processes time out during uncertainty and can communicate only with other uncertain processes .
- **Time Complexity (No Failures)**: Requires 3 message rounds (`VOTE-REQ`, vote responses, decision broadcast) . In failure cases requiring termination, unblocked sites decide within 2 additional rounds (5 rounds total) .
- **Message Complexity (No Failures)**: Uses $3n$ messages for n participants . Under worst-case cooperative termination (all n participants uncertain), message complexity reaches $n(3n + 7)/2$.

References

[1] Leslie Lamport. *LaTeX: a Document Preparation System.* Addison Wesley, Massachusetts, 2 edition, 1994.